

PJ1 实验报告

一、模型结构

$\sin(x)$ 函数本身并不复杂，通过实践，我发现两层以上的神经网络拟合效果并不好，而单层 10 个神经元的拟合效果相当不错。因此，这个网络就是一个三层的神经网络。其中，输入输出的神经元个数相同，均为采样点的个数。中间层神经元个数为 10。

对于 mnist 手写数字识别，在这次作业中不使用卷积神经网络，而是将手写数字图片展平成向量输入到全连接神经网络中，本质上是在拟合 $\sin(x)$ 的结构上的更改。

本文涉及到的内容包括：模型初始化、神经网络构建及前向传播、反向传播、参数调整（神经网络层数、loss function、可视化、学习率、学习轮数）等内容。对于激活函数的选择，我放在了 pj3 的消融实验当中。对于 batch size 的选择，本文有一定的涉及，在 pj3 中还会进一步说明。

二、模型参数细节

1、初始化

① $\sin(x)$ 函数的拟合

首先我先进行初始化。对于定义域为 -3.14 到正 3.14 的 $\sin(x)$ 函数，其图像包含了一整个周期，整体趋势为先减小后变大再减小。如果采样点过少，可能模型无法学习到这三段变化，最终结果会比较差。在训练过程中，我开始时仅仅选择了 100 个到 200 个采样点，最后拟合的图像均表现为中间的上升趋势学习较好，但是两边的减小趋势学习的不好，尤其是 -3.14 和 3.14 附近，图像并没有体现出明显的回归 $y=0$ 的趋势，反而出现了梯度较缓的样子。这可能是因为采样点较少，中间的那段上升区域涵盖 -1.57 到 1.57 ，长度为 3.14 ，相对两端长度为 1.57 的部分来说采样点更多，因此学习效果更好。

与此同时，我采用的采样方法是 `np.random.uniform`，这种方式是在 -3.14 至 3.14 之间生成随机数，并不保证随机数之间的间隔，因此较少的采样点可能会导致生成的随机数集中在某一个特定的区域。而增加采样点之后生成的随机数会比较接近均匀分布。

之后我尝试了 1000 和 2000 个采样点，发现这时模型均能够较好地学习出 $\sin(x)$ 函数的样子。但是 2000 个采样点的收敛比 1000 个采样点慢很多，在 60000 轮的训练轮次下，1000 个采样点的 loss 已经达到 0.000000，但是 2000 个采样点的 loss 仍然处在 $0.00002x$ 至 $0.00001x$ 附近。可能的原因是采样点越多，模型的参数越多，计算越复杂，局部最小值也越多，模型找到更优解的难度会增大。因此最终我采用了 1000 个采样点的方案。

在这个实验中，我并没有使用 mini batch 的形式，但是最后的拟合效果仍然很好。这可能是因为拟合 $\sin(x)$ 函数本身并不复杂的原因。

②mnist 手写数字识别

手写数字识别的文件格式比较特殊，我通过 `struct.unpack` 方式读取了图片和标签的相关信息，并用 `split` 函数将原有的数据集划分为训练集和测试集。最后用 `split_training_data` 函数将训练集划分为 `mini batch`，方便后续的训练。其余的思路与拟合 $\sin(x)$ 函数大致相同，只是改变了训练过程。具体来说每次处理一个 `mini batch`，在每一个 `mini batch` 中有多张图片 and 对应标签，将这些照片一起展平，同时输入到神经网络中，并反向传播更新模型的参数，最后 `loss function` 使用 `cross entropy` 计算 `loss`。

`mini batch` 的优点在于将原本庞大的数据集划分为小块，从而减少单次处理的负荷，方便电脑的运算。但是一次只处理一张图片的方式也不可取。在这种方式中，模型仅能学到一张图片的特征，这种特征往往不具有代表性，从而导致模型参数更新过于频繁，模型会比较不稳定，训练效果也会比较差。在实验过程中，我尝试了将数据集按 7: 3 的比例划分成训练集和测试集，以及按 8: 2 的比例划分两种方式，分别对应 112 和 128 的 `batch size`，但是在结果上，两者并没有显著的区别。具体数据为同样在 2 层，每层 20 个神经元的结构下，采用 0.001 的学习率，训练 1000 轮，两者都达到了 95% 的准确率。在后续的实验过程中，我将数据集固定按照 8: 2 的比例划分，并测试了 `batch size` 为 128 和 64 的数据，发现也都达到了 95% 的正确率。因此最后我将数据固定为按照 8: 2 的比例划分，`batch` 大小为 128 的方式。

2、神经网络构建及前向传播

为了能够灵活调整学习率、神经网络层数、神经元个数等，我希望我的神经网络能够根据不同的输入灵活搭建。在 $\sin(x)$ 的拟合中，使用者可以自主设置采样点个数、测试点个数、神经网络层数、每一层的神经元个数、学习率、学习轮数。在手写数字识别中，使用者可以选择以多少的比例拆分原始数据、`mini batch` 的大小、神经网络层数、神经元个数、学习率、训练轮数、检查频率。这种互动式输入在写代码时可能相对繁琐，但是在后续的调试过程中却显得十分便捷。

首先我初始化了 `count` 这个向量，用于记录每一层的神经元个数。接着我使用 `nn` 来记录层间的 `weight`, `bias` 参数。其中，`weight` 是一个二维矩阵，行数为更深层的神经元个数，列数为较浅层的神经元个数。`bias` 则是一个向量，大小为更深层的神经元个数。这种初始化方式符合全连接层的特点，即层与层之间的每一个神经元互相连接。而 `bias` 并不需要为每一个连接都提供，只需要在一层中添加一次即可。

我还需要专门记录神经网络的每个值。在最初版本中，我先初始化了一个全部为零的矩阵，在前向传播的过程中依次向其中加入数值。但是这种方法还需要额外的填充数值操作，也需要手动设定矩阵的具体大小，比较不便。最终我将神经网络的记录放在了前向传播中，这样计算一层就添加一层的方式十分方便。前向传播的步骤是 `weight` 乘以 `activation` 加上 `bias`。不过在拟合 $\sin(x)$ 的任务中，最后一层的映射并不是映射至 -1 到 1，因此最后一层不应该加上激活函数。而其余层都需要加上激活函数。而在手写数字识别中，我最后一层也没有用和前几层一样的激活函数，而是用 `softmax`

替换了最后一层，这样最后一层的值才能被映射到 0 和 1 之间，同时保证这些值的和为 1。

3、反向传播

对于反向传播，我初始化了一个四维矩阵。这一思想来源于我最初的简单设想：第一维表示第几层，第二维度表示这一层的神经元个数，第三维度表示更深层的神经元个数，第四层用 0, 1 分别对应 weight 和 bias。但是我对于 python 的语言特点并不十分了解，这也是我第一次使用 python 语言写较长的代码，因此在模型的搭建上显得有些笨重。后来我了解到使用(weight, bias)的数值对的形式会更加方便一些。

在计算 weight、bias 的更新值时，我开始时想要直接翻译老师下发的 pdf 文件中的参数更新公式，没有意识到矢量化优势。在第一次写代码的过程中我使用了多层 for 循环来进行反向传播中梯度的更新，但是这种做法不仅使得阅读代码变得极为困难，调试时也很难发现错误。与此同时，将多个神经元（每一层、每一批）的神经元进行整体更新，即采用矢量运算的方法会使得语言更加清晰易懂。我也了解到 numpy 库有专门针对矩阵运算的优化，因此矢量化还可以提升运算的速度。

原有的代码在 correct.py 中的注释部分中，我将它们放在了下面这张图中：

```
#this part didn't use array to store error, and forgot to spread the error
for layer in range(count.size-2,-1,-1):
    for unassigned in range(count[layer]):
        for assigned in range(count[layer+1]):
            #the special last layer
            if(layer==count.size-2):
                di=train_arr[assigned];
                oi=activation[layer+1,assigned];
                error_oi=di-oi;
                # output does not use sigmoid
                delta_matrix[layer,unassigned,assigned,0]=learning_rate*error_oi*activation[layer,unassigned];
                delta_matrix[layer,unassigned,assigned,1]=learning_rate*error_oi;
            else:
                for assigned_p in range(count[layer+2]):
                    weight,_nn_parameters[layer+1];
                    wji=weight[assigned_p,assigned];
                    #error/oi
                    if(layer==count.size-2):
                        di=train_arr[assigned];
                        oi=activation[layer+1,assigned];
                        error_oi=di-oi;
                        delta_i=error_oi;
                    else:
                        error_oi=delta_matrix[layer+1,assigned,assigned_p,1]/learning_rate;
                        oi=activation[layer+2,assigned_p];
                        delta_i=error_oi*(1-oi)*oi;
                    error_sum+=wji*delta_i;

                ok=activation[layer,unassigned];
                oj=activation[layer+1,assigned];
                delta_j=error_sum*oj*(1-oj);
                delta_matrix[layer,unassigned,assigned,0]=learning_rate*delta_j*ok;
                delta_matrix[layer,unassigned,assigned,1]=learning_rate*delta_j;
```

这次经历给了我一些启示：

- 1) 能够矢量化尽量抛弃 for 循环
- 2) 即使需要多个 for 循环，也应该使用有意义的变量名称，如 assigned 表示已

经计算好的，对应深层；unassigned 表示尚未计算好的，对应浅层。使用 i, j, k 等变量容易混淆它们表达的含义，造成不便。

- 3) 计算过程并不需要一步到底。相反地，给每一个步骤一个明确的目标，最后再将它们乘起来会显得程序较为简洁清晰。具体来说，可以使用 error 表示误差项，体现的是数学上的偏导，这样再分别乘以学习率及激活值就显得条理更加清晰了。

4、训练及测试：

1) loss function

①sin(x)函数的拟合

最终的 loss function 使用的是 MSE 均方误差，这种误差函数与课堂中讲述的一致。MSE 相对于平均绝对误差来说，最大的优势在于它是连续可导的，这是梯度下降法成立的前提条件。而选择了 MSE 之后才能确定后续的反向传播的参数更新方式。

②手写数字识别

这是分类任务，因此我选择 cross entropy 作为 loss function。具体实现方式是将十种分类转换为十维向量，用 0 至 1 之间的值表示可能性。但是在实验初期，我忘记更换了 loss function，使用了 MSE 计算 loss，这种方式并不合理，因为分类任务的 loss 并不是简单地衡量每个值与 1 或 0 之间的差距，关键在于模型认为最为准确的值是否与标准答案一样。这也是为什么分类任务中人们会用 top-1 error 和 top-5 error 来衡量拟合效果，前者是只看模型认为的最可能的预测类别，后者是看模型认为前五可能的预测类别。虽然使用 MSE 也可以看出 loss 在减少，但是相较于单纯地衡量数值上的差距，针对分类任务的 cross entropy 才是更合适的 loss function。

这是我做实验时的疏忽之处。经过这个错误，我意识到了 loss function 的重要性。loss function 虽然并没有在我的代码中被显式写出来，但是没有 loss function 就没有参数更新的规则，而参数更新的过程中其实体现了 loss function 本身。因此在实验过程中，应该先确定好 loss function。

2) 可视化

①sin(x)函数的拟合

在训练过程中，我设置了每 100 轮显示一次 loss，并画出了模型训练完成后的 sin(x)函数与正确的 sin(x)函数，通过二者的重合程度可以非常直观地看出拟合的效果。

②手写数字识别

由于训练轮数远远小于拟合 sin(x)的六万轮，为了使得最终的曲线更加光滑连续，我选择每 20 轮显示一次 loss，最终以正确率的方式来衡量模型的学习成果。

3) 调整网络结构

① $\sin(x)$ 函数的拟合

在开始时，我仅仅训练了 3000 轮，这样的训练轮数是不正确的。但当时我对于正常的训练量级没有明确的感受。于是我想到了两种思路：

第一种是调整层数。虽然老师上课说过拟合 $\sin(x)$ 只用一层就够了，但我还是在迷茫之中尝试了增加层数的方法。我猜测 $\sin(x)$ 有多段转折，并且并不是简单的线性形式，单层的神经网络可能不足以捕捉这些信息。但是我发现，在其他参数（采样点=1000，学习率=0.1，训练轮数=3000）不变的情况下，采用两层神经元，每层神经元个数为 10 时出现了严重的梯度爆炸问题。考虑到一共 20 个神经元可能太多，我将网络调整为两层，每层三个神经元的形式，拟合效果仍然不佳。

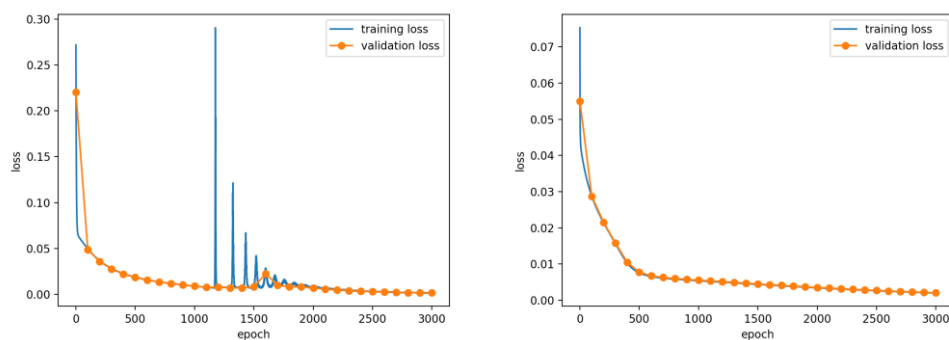


图 1：左图为两层，每层神经元个数为 5 的网络，右图为一层，神经元个数为 10 的网络。二者均在采样点为 1000 个，学习率为 0.1 的情况下训练 3000 轮，

由于两层的神经网络相比一层的多了一次传播梯度的过程，因而梯度更加容易出现爆炸的情况。于是我保留了一层的网络结构，并尝试了第二种思路：调整学习率。首先我从较大的学习率开始尝试。在 $\text{learning rate}=0.15$ 时，依然在较浅层就出现了梯度爆炸问题。我又尝试了较小的学习率 0.02，但是经过一万轮的训练之后在测试时 loss 仍然为 0.05，效果一般。这种情况的原因是：学习率过高时，尽管模型调整参数的幅度变大了，但是参数相对也较大，从而会产生梯度爆炸，进而导致模型学习效果不稳定，甚至可能因为浮点数溢出而无法学习，导致效果变差。而学习率过小会导致模型更新参数的幅度过小，使得模型训练变得很慢，也可能使得模型卡在局部最优解，无法找到更优解，导致学习效果的降低。我继续尝试学习率，发现在 0.1 以上时梯度在较浅层仍然会出现爆炸问题。而在 0.09 时，模型在训练了 3 万轮之后仍然没有出现梯度爆炸问题。于是我将学习率固定为 0.09。

②手写数字识别

在开始时我仍然尝试了 1 层 10 个神经网络的做法，但是在学习率为 0.001，训练 500 轮的情况下正确率没有到达 90%。于是我增加了神经元个数，发现在 20 个和 30

个神经元的时候训练 500 轮正确率能够到达 92%。经过一系列的调试，我发现 1 层神经网络在训练轮数增多的情况下效果和两层神经网络的差不多，比如在相同的 0.001 学习率的情况下，1 层 30 个神经元的神经网络在训练了 3000 轮后达到了最好结果

(96.7%)，但是平均下来 2 层的神经网络的准确率显得更高。因此最终我采用了两层每层 40 个神经元的架构，并增加了训练轮数至 2000 轮，达到了 96.7% 的正确率。

4) 训练轮数

① $\sin(x)$ 拟合

将学习率设定为 0.09 后，模型在训练 3 万轮后在测试中已经得到了 0.000003 的 loss。因此我进一步调大了学习轮数至 6 万轮。在实验中，模型在大约 46000 轮的时候将 loss 降低至 0.000000，最终的拟合图如下：

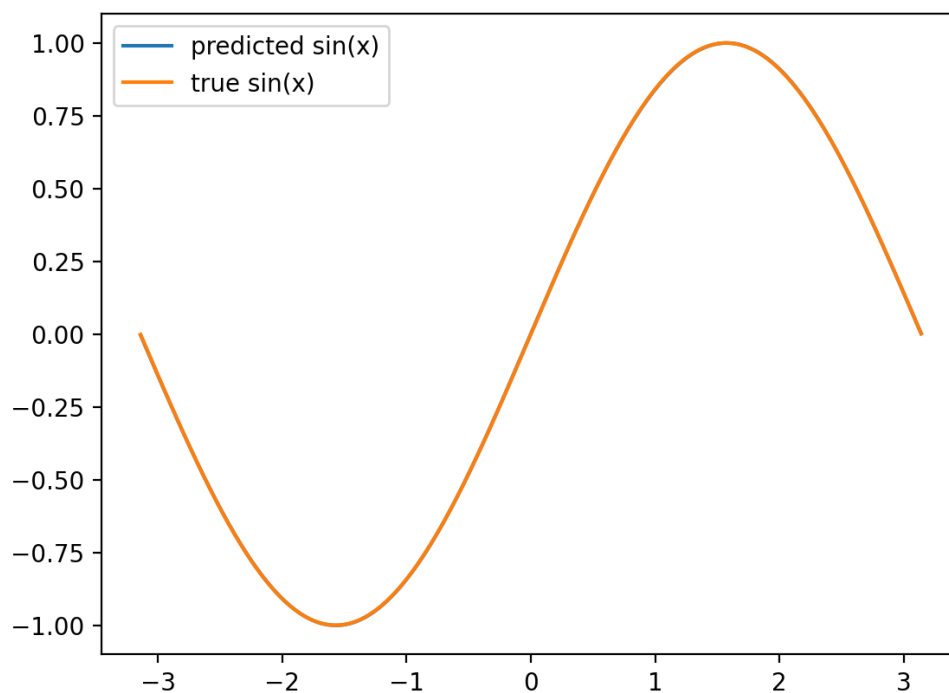


图 2：最终参数调整到最佳时的拟合效果图

但是我在多次实验之后发现，虽然 0.09 的学习率能够学习到比较好的效果，但是在训练轮数增多之后，或者是模型随机采样导致初始点不同都有可能产生梯度爆炸的情况。但是这种情况在训练轮数增大之后最终都得到了理想的拟合效果。原因可能有这两点：

- 1) 虽然图像中可以看到梯度爆炸的现象，但是这种幅度变化相比学习率很大的时候要小很多，并不一定超出了浮点数上限，模型仍然能够通过学习进一步优

化。

- 2) 在短暂的梯度爆炸之后，模型到达了更优的局部最小值，使得模型的表现逐渐稳定，并保持到最后。

②手写数字识别

相较于拟合 $\sin(x)$ 函数，手写数字识别的输入不再是单纯的向量，而是 $28*28$ 的几千张图片，计算量上来说要高很多。因此在初期，我从 500 轮开始尝试，但是发现训练效果并不好。再多次调整参数之后我发现，两千轮以上的拟合效果才会比较好，因此我选择了训练三千轮。

具体参数：

	激活函数	ratio	batch_size	latent	neuron	r	epoch	train_loss	val_loss	acc
1	tanh	0.8	128	1	10	0.001	500	0.039612	0.039981	0.88275
2	ReLU	0.8	128	1	20	0.001	1000	0.024987	0.025881	0.925417
3	ReLU	0.8	128	1	30	0.001	500	0.02369	0.025156	0.928417
4	ReLU	0.8	128	1	10	0.001	1000	0.023214	0.024424	0.9235
5	tanh	0.8	128	2	10,10	0.001	300	0.039646	0.040311	0.879667
6	tanh	0.8	128	1	10	0.1	300	0.078074	0.078216	0.730333
7	tanh	0.8	128	1	10	0.0001	500	0.084268	0.084601	0.815
8	tanh	0.8	128	1	10	0.001	500	0.038627	0.039648	0.876333
9	ReLU	0.8	128	1	40	0.001	500	0.023069	0.023402	0.934083
10	ReLU	0.8	128	1	10	0.001	300	0.029635	0.029942	0.914333
11	ReLU	0.8	128	1	10	0.001	1000	0.023501	0.024746	0.92375
12	ReLU	0.8	128	1	40	0.001	500	0.021798	0.021764	0.94075
13	ReLU	0.8	128	2	10,10	0.001	500	0.023555	0.024335	0.927083
14	ReLU	0.8	128	2	20,20	0.001	1000	0.011999	0.014534	0.951083
15	ReLU	0.8	128	2	30,30	0.001	1000	0.009437	0.012127	0.960333
16	ReLU	0.8	128	2	40,40	0.001	2000	0.008742	0.010984	0.963583
17	ReLU	0.8	128	3	10,10,10	0.001	1000	0.016235	0.018377	0.939083
18	ReLU	0.8	128	2	40,40	0.001	1000	0.004348	0.008742	0.9655
19	ReLU	0.8	128	2	10,10	0.001	500	0.024191	0.025233	0.922833
20	以上的loss是mae									
21	以下的loss是cross-entropy loss									
22	ReLU	0.7	112	2	20,20	0.001	1000	1.518559	1.52739	0.9545
23	ReLU	0.8	128	2	30,30	0.001	2000	1.487167	1.50516	0.964
24	ReLU	0.8	64	2	20,20	0.001	1000	1.501897	1.519323	0.954916
25	ReLU	0.7	112	2	20,20	0.001	1000	1.517065	1.524405	0.955111
26	ReLU	0.7	112	2	20,20	0.001	1000	1.515304	1.526503	0.953611
27	ReLU	0.8	128	2	20,20	0.001	2000	1.498	1.519987	0.953416
28	ReLU	0.8	128	1	40	0.001	2000	1.504699	1.516522	0.962167
29	ReLU	0.8	128	1	30	0.001	2000	1.510097	1.518988	0.962167
30	ReLU	0.8	128	1	30	0.001	3000	1.493096	1.507229	0.967333
31	ReLU	0.8	128	2	40,40	0.001	500	1.523313	1.529108	0.956833

三、 对反向传播的理解：

在反向传播中，模型通过将前向传播的结果与正确结果进行比较，代入 loss function 计算得出在当前参数下模型的 loss，并通过对 loss function 求导得到更新参数的方式。这种方式称作梯度下降法，目标是找到 loss function 所代表的高维函数的最小值，从而实现最高的准确率。尽管 loss function 是高维函数，在高维空间中仍然可以用梯度这一特征来刻画函数的变化规律：逆着梯度变化的方向，函数的值减小，顺着梯度变化的方向，函数的值变大。因此，通过求偏导的方式就可以确定每一个参数应该向着哪个方向更新的数值大小。通过这种方式，loss function 的值会逐步减小，故而被

称作“梯度下降”法。

然而，loss function 是非凸函数，局部最小值并不一定是全局最小值。因此，需要人为设定函数更新梯度的步幅，从而让函数能够从一个局部最小值“跨”出去，找到新的局部最小值。于此同时，如果函数更新梯度的步幅过大，模型可能无法到达局部最小值，而是在一定范围内左右徘徊，这种现象需要得到限制。达到这两个目的参数便是学习率。学习率与普通的求导结果之相乘之后可以控制模型参数改变的幅度，从而让模型的参数更新更加稳定。

反向传播之所以叫做“传播”，是因为偏导的链式法则。这个法则使得更深层的偏导结果可以被用于计算更浅层的偏导结果，因此深层的偏导结果被作为一个参数向浅层传递，达到了减小计算量的效果。这种传递方式会一层层向浅层传递，从而更新整个神经网络的参数。